

Macros programming

Introduction

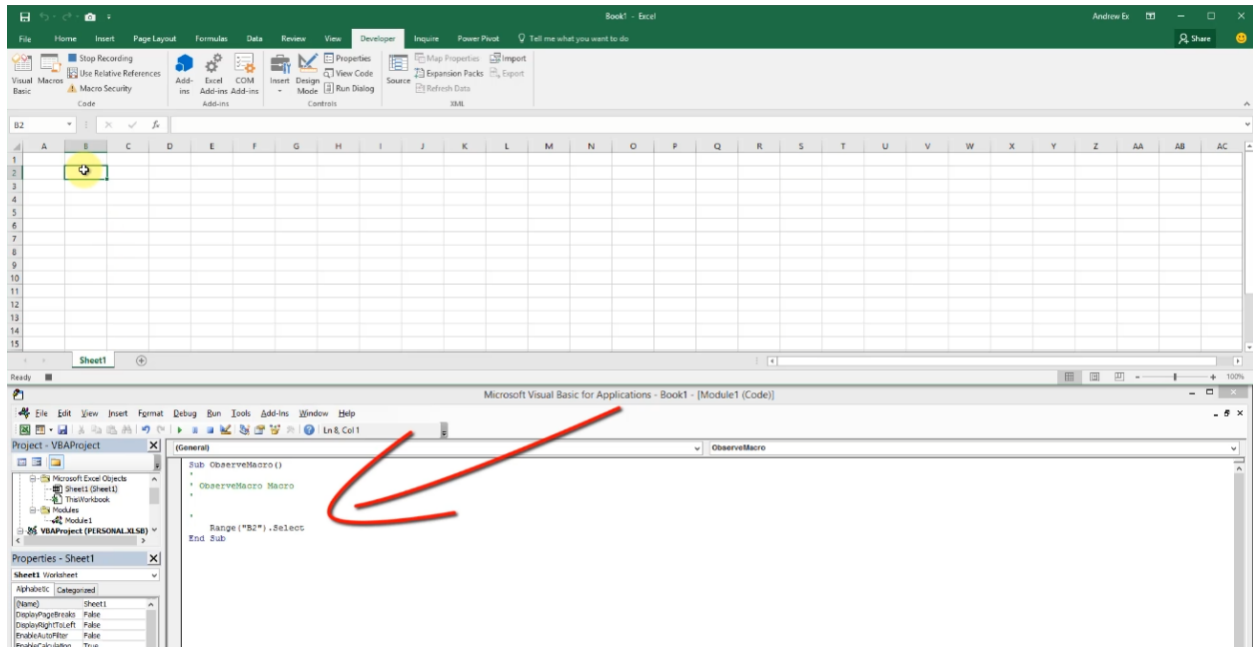
As we have seen in the VBE, Macros are written using VBA, a programming language developed by Microsoft. It is used to automate tasks and create custom applications in Microsoft Office applications, including Excel, Word, PowerPoint, Access, and Outlook.



VBA is a version of Visual Basic that is customized for use with Office applications. In this course, we will focus only on Excel to extend its functions. VBA code can be used to create custom functions and perform complex calculations.

VBA is a flexible and powerful tool that can be used by both beginners and advanced users. You already know that when you record a macro, Excel is writing the associated VBA code.

Inside the VBE, one can open or create a module using the Project Explorer. Also, you can open the VBE and start recording a macro to view how the VBA code is being generated.



Before we continue, let's talk a little about macro recording vs macro programming.



The main difference is the level of control that you have over the resulting macro.

As we have explained before, macro recording involves recording a series of your actions in Excel. You simply perform the desired actions and Excel records those actions as VBA instructions.

On the other hand, macro programming involves writing VBA code to create more complex macros that can perform a wider range of tasks. Macro programming requires a deeper understanding of the VBA programming language and typically involves writing code in the VBE to create the macro.

While both ways are correct ways to create a macro, let's see their pros and cons:

Macro recording	Macro programming
Pros: Records every task Programming knowledge is not required	Pros: Allows a lot of control over the program Can loop through a range of data Can create more advanced and complex applications
Cons: Cannot loop through a range of data Uses a lot more code than is required	Cons: Requires programming knowledge and experience Requires more time to create a macro

Let's see a macro for changing the orientation of a worksheet:

By using the macro recording, we will get the following macro:

```
Sub Macro1()  
Application.PrintCommunication = False  
With ActiveSheet.PageSetup  
    .PrintTitleRows = ""  
    .PrintTitleColumns = ""  
End With  
Application.PrintCommunication = True  
ActiveSheet.PageSetup.PrintArea = ""  
Application.PrintCommunication = False  
With ActiveSheet.PageSetup  
    .LeftHeader = ""  
    .CenterHeader = ""  
    .RightHeader = ""  
    .LeftFooter = ""  
    .CenterFooter = ""  
    .RightFooter = ""  
    .LeftMargin = Application.InchesToPoints(0.7)  
    .RightMargin = Application.InchesToPoints(0.7)  
    .TopMargin = Application.InchesToPoints(0.75)  
    .BottomMargin = Application.InchesToPoints(0.75)  
    .HeaderMargin = Application.InchesToPoints(0.3)  
    .FooterMargin = Application.InchesToPoints(0.3)  
    .PrintHeadings = False  
    .PrintGridlines = False  
    .PrintComments = xlPrintNoComments  
    .PrintQuality = 600  
    .CenterHorizontally = False  
    .CenterVertically = False  
    .Orientation = xlLandscape  
    .Draft = False  
    .PaperSize = xlPaperLetter  
    .FirstPageNumber = xlAutomatic  
    .Order = xlDownThenOver  
    .BlackAndWhite = False  
    .Zoom = 100  
    .PrintErrors = xlPrintErrorsDisplayed  
    .OddAndEvenPagesHeaderFooter = False  
    .DifferentFirstPageHeaderFooter = False  
    .ScaleWithDocHeaderFooter = True  
    .AlignMarginsHeaderFooter = True  
    .EvenPage.LeftHeader.Text = ""  
    .EvenPage.CenterHeader.Text = ""  
    .EvenPage.RightHeader.Text = ""  
    .EvenPage.LeftFooter.Text = ""  
    .EvenPage.CenterFooter.Text = ""  
    .EvenPage.RightFooter.Text = ""  
    .FirstPage.LeftHeader.Text = ""  
    .FirstPage.CenterHeader.Text = ""  
    .FirstPage.RightHeader.Text = ""  
End With  
Application.PrintCommunication = True
```

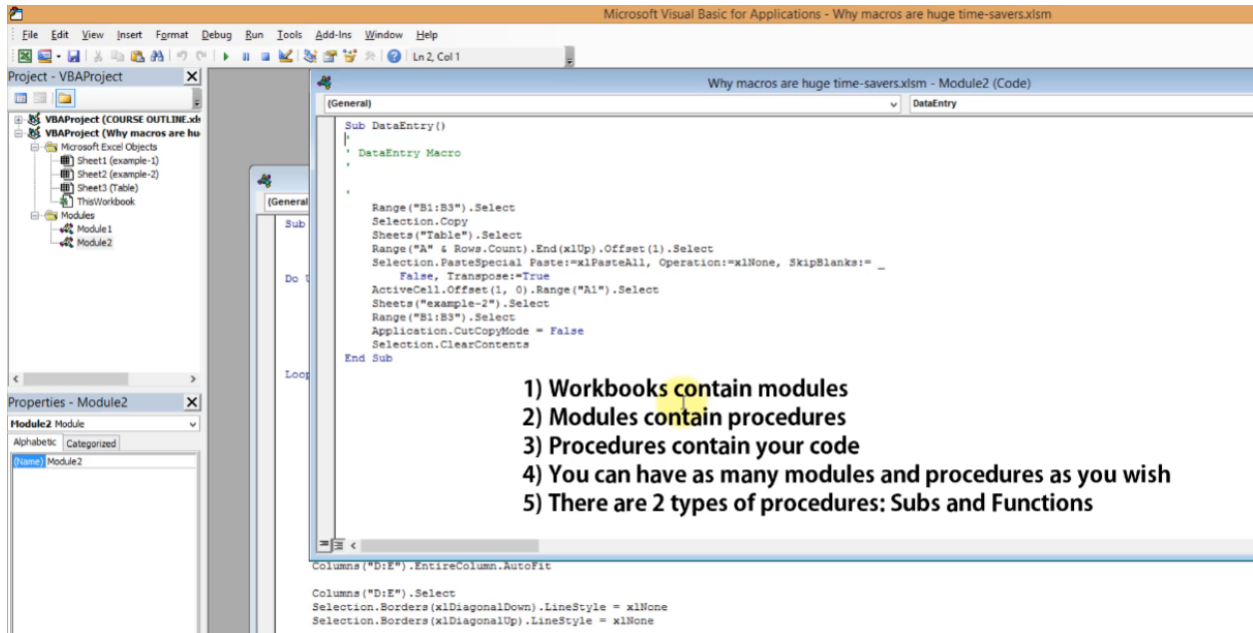
On the other hand, once we know macro programming, the same macro will have the following form:

```
Sub Macro1()  
    ActiveSheet.PageSetup.Orientation = xlLandscape  
End Sub
```

One recommendation that we can tell you is to use the best of both worlds. In other words, with macro programming, you can get help from macro recording!

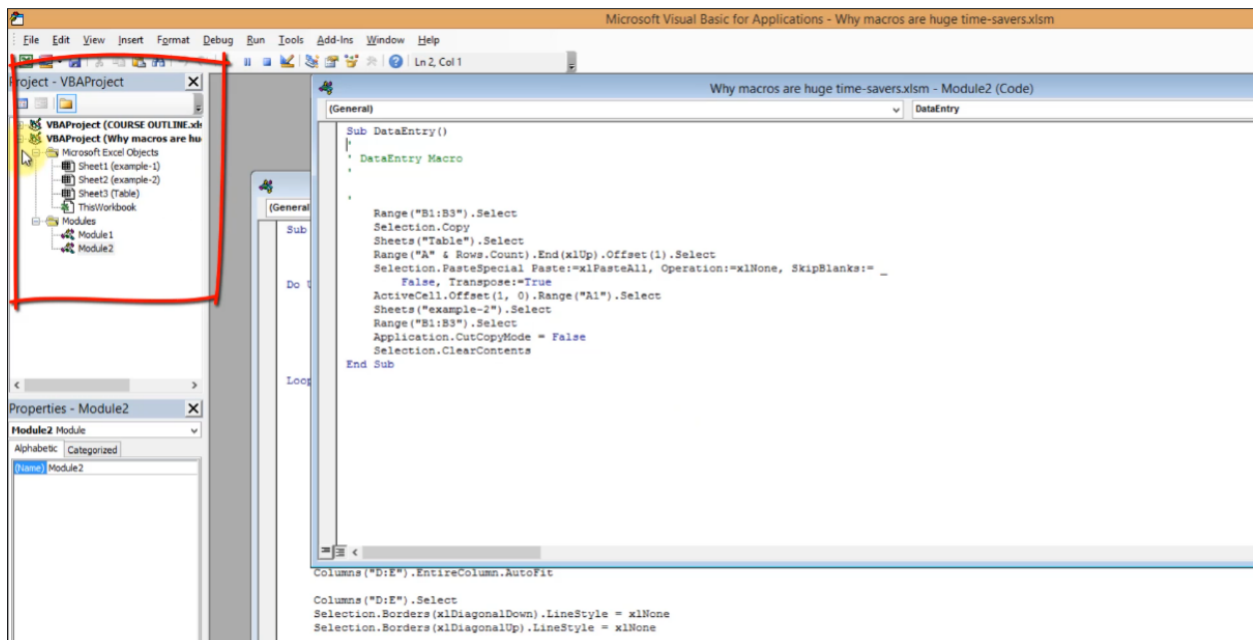
Macros Programming II

Let's continue covering concepts related to macro programming with VBA and the VBE.



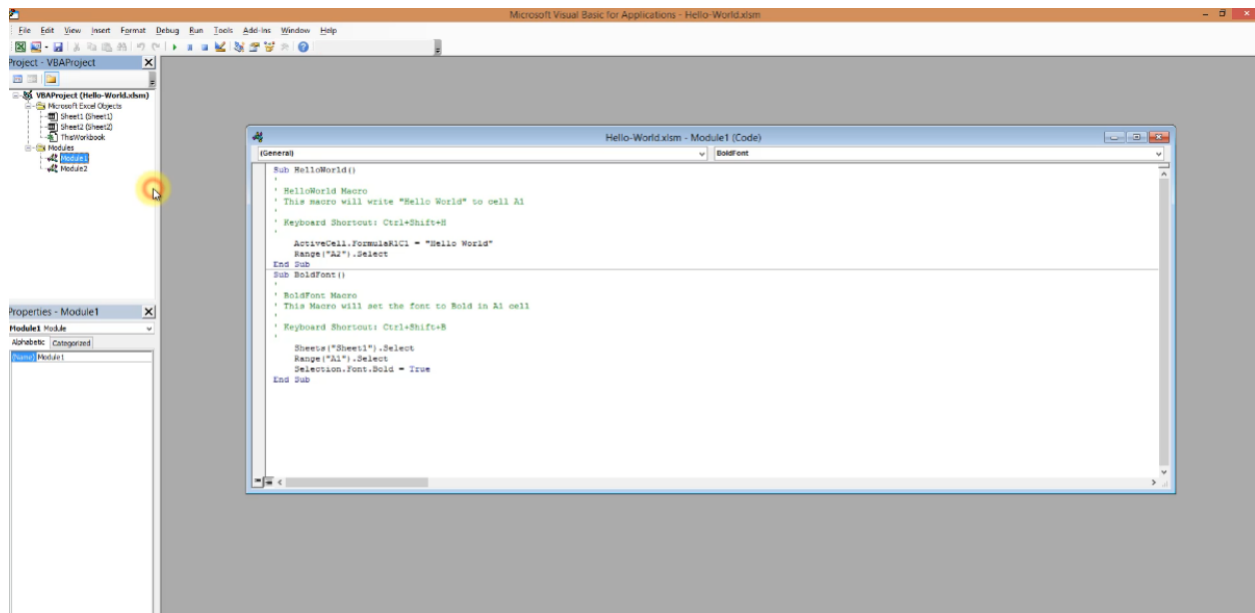
In VBE, the project window displays a list of all open workbooks and add-ins. This list is displayed as a tree diagram. When you double-click a module, the code in the module appears in the code window. Previous stored or recorded modules can be found in the current workbook.

When you record a macro, Excel automatically inserts a VBA module to hold the recorded code.



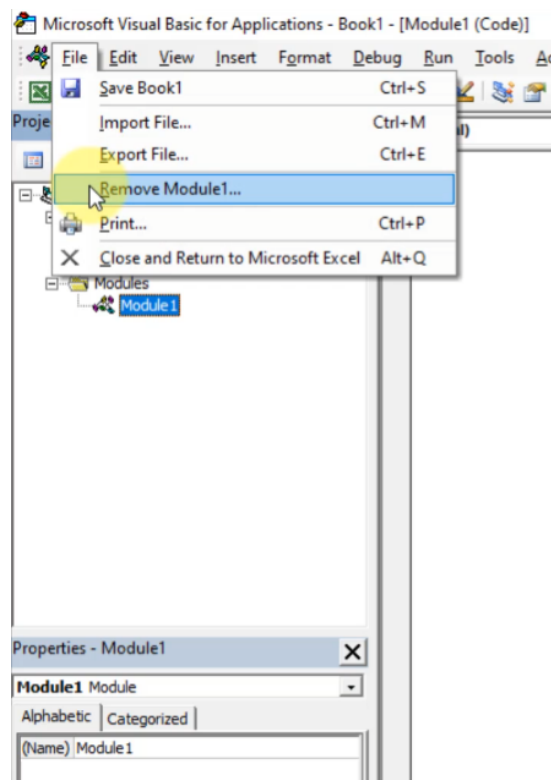
The recorded macro is a sub-procedure. When you run a macro, it goes through the lines of code, usually called 'statements', inside the sub-procedure in the order in which they appear. So, it performs the task given by the first statement, and when that statement is finished, it moves on to the next one and performs the next task.

Each module can have multiple macros on it. Sometimes, it's useful to create a module per macro or a module that contains macros that perform similar actions. Remember that you can cut the code from a module and paste it into another.

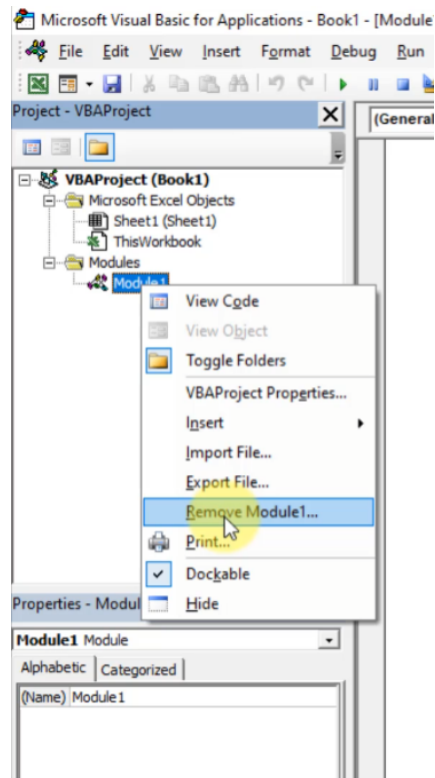


Sometimes you will need to remove a VBA module from a project. For example, it may contain code that you no longer need, or it's empty because you recorded a macro and then changed your mind.

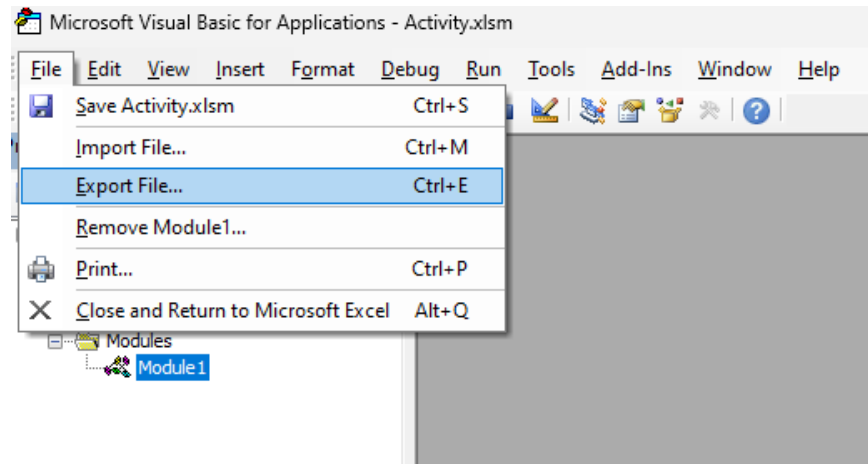
To remove a VBA module from a project, select the module name in the Project Explorer and go to the File menu and select 'Remove X', where X is the module name.



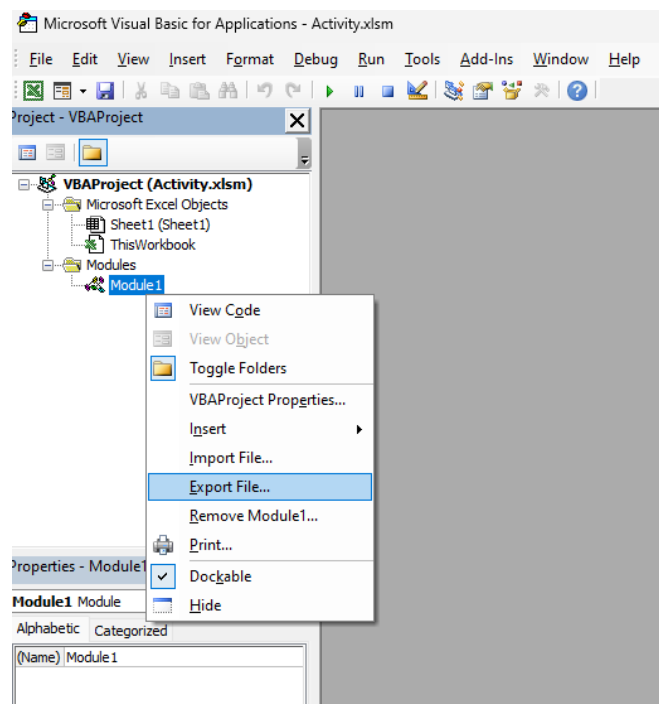
Another way to remove a module is to right-click the module's name and choose to remove the module.



Remember when we talk about '.bas' files? As we mentioned, a module can be exported as a '.bas' file for sharing or using the macros contained on it in other macro-enabled workbooks. To export a module, select it and go to the File menu and select the 'Export File' option.



You can also select the module you want to export, right-click it, and select the 'Export File' option.



Macros programming – The programmer's way

So far in the previous two sections, we have talked about macros programming as concepts and its functionalities, but! We haven't started coding!



As with any other programming language, VBA includes:

1. Variables.
2. Data types.
3. Operators.
4. Control structures.
5. Procedures: Procedures are used to group a set of instructions to perform a desired task. In VBA, procedures can be either Sub procedures (which do not return a value) or Function procedures (which return a value). More on this later!
6. Objects: VBA is an object-oriented programming language, which means that it uses objects to represent data and functionality. Objects have properties (attributes that describe the object) and methods (actions that can be performed on the object).
7. Events: They are actions that occur in response to a user action, such as clicking a button or opening a file. VBA allows you to write code that responds to specific events in an application.

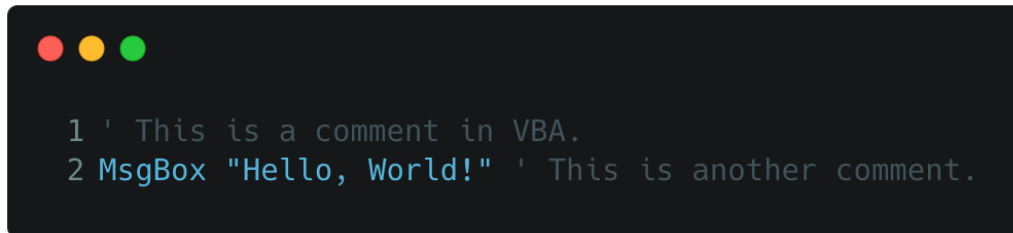
Sometimes you will need to check the software reference page. It is a written description of how it was designed, organized, and how it operates. In other words, it provides a collection of documents that describe its features, functionality, architecture, and how you can use it. The following link will take you to the Excel VBA reference website:

[Excel Visual Basic for Applications \(VBA\) reference | Microsoft Learn](#)

Additionally, in VBA you can add comments! Comments in programming are lines of text that are added to the code to explain what the code does.

They are used to make your code more understandable for other programmers who may read it later or for your future self if you need to review the code later.

In VBA, comments can be added by using the apostrophe character (') at the beginning of a line. Any text that follows the apostrophe on the same line is considered a comment and is ignored by the compiler.



```
1 ' This is a comment in VBA.  
2 MsgBox "Hello, World!" ' This is another comment.
```

In the above example, the first line is a comment that explains that it is a comment in VBA. The second line displays a message box with the text "Hello, World!" and the last part of the line is another comment that explains what the line of code does.

Variables and Data Types

Variables are used to store data and are a fundamental element of any programming language.

To declare a variable in VBA, you must use the 'Dim' keyword followed by a name. This name in programming is called an identifier. It is a unique name to identify specific data inside your code.

The rules for naming a variable identifier in VBA are:

1. The first character must be a letter (A-Z or a-z).
2. You cannot use a space, hyphen (-), period (.), or any special character in the name (!, @, &, \$, #).
3. The maximum length of a variable identifier is 255 characters.
4. Avoid reserved keywords as names. A reserved keyword is a word that is already used within VBA functionality. Usually, VBE will highlight any reserved keyword that you use but always double-check!
5. You cannot repeat names and variables are case-insensitive, meaning that "myVariable" and "MyVariable" are considered the same variable.
6. Try to avoid an underscore (_). Even though it is not prohibited, it could cause problems when running the program.

```
1 Dim number1
2 Dim positionX
3
4 Dim my_position ' Ok, but could generate an errors
5
6 Dim myVariable
7 ' The following variable will cause an error
8 ' as it has the same "name" (case-insensitive)
9 Dim myvariable
10
11 ' Variable's identifiers not permitted!
12 Dim 8names
13 Dim chic@s
14 Dim power!
15 Dim h&m
```


Additionally, VBA supports various data types. They specify the type of data that a variable can hold. The following list shows the different data types considered in VBA:

1. Integer: Used for whole numbers (e.g. 1, 2, 3). Integers have a range of -32,768 to 32,767.
2. Long: Used for larger whole numbers (e.g. 1000, 10000, 100000). Longs have a range of -2,147,483,648 to 2,147,483,647.
3. Single: Used for single-precision floating-point numbers (e.g. 3.14, 2.5, 0.01). Singles have a range of approximately -3.402823E38 to 3.402823E38.
4. Double: Used for double-precision floating-point numbers (e.g. 3.14159, 2.71828, 0.0001). Doubles have a range of approximately -1.79769313486232E308 to 1.79769313486232E308.
5. String: Used for text data (e.g. "Hello World!", "12345"). Strings have a maximum length of approximately 2 billion characters.
6. Boolean: Used for True/False values.
7. Date: Used for dates and times (e.g. 1/1/2022, 12:30:00 PM).

```
1 Dim myInteger As Integer
2 Dim myString As String
3 Dim myBoolean As Boolean
4
5 ' Therefore, the variables can store only the specified data type
6
7 myInteger = 5
8 myString = "VBA is awesome!"
9 myBoolean = False
```

Now, a related concept in this topic is constant variables.



A constant is a variable whose value remains the same throughout the execution of the program. Once a constant value is assigned to a variable, it cannot be changed during the program's execution.

Constants are useful for storing values that should not be modified, such as mathematical or physical constants, or values that are used repeatedly in the program, usually defined by the programming language.

To declare a constant in VBA, you use the 'Const' keyword followed by the variable name and its value, as shown in the example below:

```
1 Sub circleArea()  
2  
3     Const pi As Double = 3.14159  
4     Dim radius As Integer  
5     Dim area As Double  
6     Dim resultsMessage As String  
7  
8     radius = 5  
9  
10    resultsMessage = "Area of circle with R = " & radius & " is "  
11  
12    area = pi * radius ^ 2  
13  
14    resultsMessage = resultsMessage & area  
15  
16    MsgBox resultsMessage  
17  
18 End Sub
```

In the example, we're calculating the area of a circle with a radius of 5. The result of this macro will open a pop-up window (using the command MsgBox) and show you the result of the operation.

There are more types of data that you could use in VBA. In this course, we will cover some according to our needs. You can check the reference site of Excel to know more about them:

[Data type summary | Microsoft Learn](#)

Operators

Whenever you are working with data, you must perform operations to manipulate it. VBA supports a variety of operators, such as:

a. Arithmetic operators.

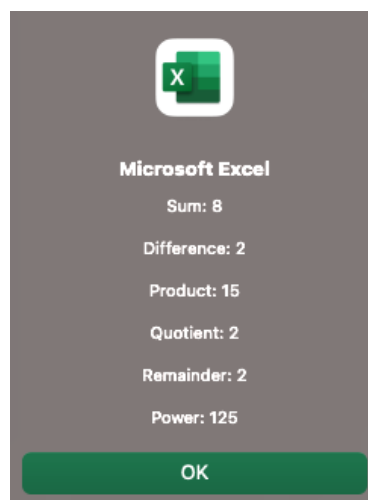
They are used to perform basic arithmetic operations on numeric values or variables. The following arithmetic operators are available:

- + (Addition): Adds two numeric values or variables and returns their sum.
- - (Subtraction): Subtracts one numeric value or variable from another and returns the difference.
- * (Multiplication): Multiplies two numeric values or variables and returns their product.
- / (Division): Divides one numeric value or variable by another and returns their quotient.
- Mod (Modulus): Divides one numeric value or variable by another and returns the remainder.
- \ (Integer Division): Divides one numeric value or variable by another and returns the quotient of the division without the remainder. For example, $7 \setminus 3 = 2$
- ^ (Exponentiation): Raise a base number to a power. In other words, it is the process of multiplying a number by itself a certain number of times.

In the following example, we will use integer values to perform all the arithmetic operations; however, you can use any other numeric value.

```
1 Sub arithmeticOperations()  
2   Dim sum As Integer  
3   Dim difference As Integer  
4   Dim product As Integer  
5   Dim quotient As Integer  
6   Dim remainder As Integer  
7   Dim power As Integer  
8   Dim resultsMessage As String  
9  
10  Dim a As Integer  
11  Dim b As Integer  
12  
13  a = 5  
14  b = 3  
15  
16  sum = a + b  
17  difference = a - b  
18  product = a * b  
19  quotient = a / b  
20  remainder = a Mod b  
21  power = a ^ b  
22  
23  resultsMessage = "Sum: " & sum & vbCrLf & _  
24                  "Difference: " & difference & vbCrLf & _  
25                  "Product: " & product & vbCrLf & _  
26                  "Quotient: " & quotient & vbCrLf & _  
27                  "Remainder: " & remainder & vbCrLf & _  
28                  "Power: " & power  
29  MsgBox resultsMessage  
30 End Sub
```

In this example, 'vbCrLf' is a constant variable of VBA that will add a break-line or 'Enter' character.



b. Comparison operators

They are used to compare two values and return a Boolean value (True or False) based on the result of the comparison. The following comparison operators are available:

- = (Equal to): Returns true if the values on both sides are equal, and false otherwise.
- <> (Different than): Returns true if the values on both sides are not equal, and false otherwise.
- > (Greater than): Returns true if the value on the left side is greater than the value on the right side, and false otherwise.
- < (Less than): Returns true if the value on the left side is less than the value on the right side, and false otherwise.
- >= (Greater than or equal to): Returns true if the value on the left side is greater than or equal to the value on the right side, and false otherwise.
- <= (Less than or equal to): Returns true if the value on the left side is less than or equal to the value on the right side, and false otherwise.

In the following example, we will use integer values to perform all the comparison operations.

```
1 Sub comparissonOperators()  
2  
3     Dim num1 As Integer  
4     Dim num2 As Integer  
5     Dim result1 As Boolean  
6     Dim result2 As Boolean  
7     Dim result3 As Boolean  
8     Dim result4 As Boolean  
9     Dim result5 As Boolean  
10    Dim result6 As Boolean  
11    Dim resultMessage As String  
12  
13    resultMessage = " "  
14    num1 = 5  
15    num2 = 3  
16  
17    result1 = IIf(num1 = num2, True, False)  
18    result2 = IIf(num1 <> num2, True, False)  
19    result3 = IIf(num1 < num2, True, False)  
20    result4 = IIf(num1 > num2, True, False)  
21    result5 = IIf(num1 <= num2, True, False)  
22    result6 = IIf(num1 >= num2, True, False)  
23  
24    resultMessage = "num1: " & num1 & vbCrLf & "num2: " & num2 & vbCrLf & "num1 = num2: " &  
    result1 & vbCrLf & "num1 <> num2: " & result2 & vbCrLf & "num1 < num2: " & result3 & vbCrLf &  
    "num1 > num2: " & result4 & vbCrLf & "num1 <= num2: " & result5 & vbCrLf & "num1 >= num2: " &  
    result6  
25  
26    MsgBox resultMessage  
27  
28 End Sub
```

The comparison operators are used usually in conjunction with control structures. In this case, we are using the ternary if operator control structure. By using the method 'IIf()' function, we will set the value of a variable according to the condition.

The result of this macro will open a pop-up window and show only the conditions that were met.



Microsoft Excel

num1: 5

num2: 3

num1 = num2: False

num1 <> num2: True

num1 < num2: False

num1 > num2: True

num1 <= num2: False

num1 >= num2: True

OK

c. Logical operators

They are used to perform logical operations on Boolean values (True or False). The following logical operators are available:

- 'And': Returns true if both operands are true, and false otherwise.
- 'Or': Returns true if either operand is true, and false otherwise.
- 'Not': Returns the opposite of the operand; if the operand is true, it returns false, and if the operand is false, it returns true.

In the following image, you can find the truth tables of the logical operators. A truth table is a table that shows the possible values of logical variables (typically represented as True or False) for each possible combination of inputs or conditions in a logical expression.

And Truth Table:

Input 1	Input 2	Output
True	True	True
True	False	False
False	True	False
False	False	False

Or Truth Table:

Input 1	Input 2	Output
True	True	True
True	False	True
False	True	True
False	False	False

Not Truth Table:

Input	Output
True	False
False	True

Truth tables are commonly used in logic, mathematics, electronics, computer science, and other fields to analyze and simplify complex logical expressions.

In the following example, we will use string values to perform all the logical operations.

```
1 Sub IsEmployeeName()  
2  
3     Dim name As String  
4     Dim lastName As String  
5     Dim resultsMessage As String  
6  
7  
8     name = "John"  
9     lastName = "Appleseed"  
10    resultsMessage = "Fullname: " & name & " " & lastName & vbCrLf  
11  
12    Dim response As Boolean  
13  
14    response = name = "Jane" Or lastName = "Appleseed"  
15    resultsMessage = resultsMessage & "Jane or Appleseed: " & response & vbCrLf  
16  
17    response = name = "Jane" And lastName = "Appleseed"  
18    resultsMessage = resultsMessage & "Jane and Appleseed: " & response & vbCrLf  
19  
20    response = name = "John" And lastName = "Appleseed"  
21    resultsMessage = resultsMessage & "John and Appleseed: " & response & vbCrLf  
22  
23    response = Not name = "Jane"  
24    resultsMessage = resultsMessage & "not Jane: " & response & vbCrLf  
25  
26    response = Not name = "John"  
27    resultsMessage = resultsMessage & "not John: " & response & vbCrLf  
28  
29    MsgBox resultsMessage  
30  
31 End Sub
```

The result of this macro will open a pop-up window and show only the conditions that were met.

